

Rice's Theorem

A Formal Mathematical Proof

Theory of Computation

1. Preliminaries

Let us define the foundational components:

- Let M denote a Turing Machine.
- Let $\langle M \rangle$ be the string encoding of the machine M .
- Let $L(M)$ be the language recognized by M .
- Let $\mathcal{RE}$ be the class of all Turing-recognizable (Recursively Enumerable) languages.

2. Defining a "Property"

Mathematically, a property of a Turing-recognizable language is a subset of $\mathcal{RE}$.

Property Definition

Let $\mathcal{P} \subseteq \mathcal{RE}$.

If $L \in \mathcal{P}$, we say L has property $\mathcal{P}$.

We can define a language corresponding to this property:

$$L_{\mathcal{P}} = \{\langle M \rangle \mid L(M) \in \mathcal{P}\}$$

The goal is to determine if a decider exists for $L_{\mathcal{P}}$.

3. Trivial vs. Non-Trivial Properties

- **Trivial Property:** $\mathcal{P} = \emptyset$ or $\mathcal{P} = \mathcal{RE}$.
 - ▶ The property applies to no languages, or to all languages.
 - ▶ $L_{\mathcal{P}}$ is trivially decidable.
- **Non-Trivial Property:** $\mathcal{P} \neq \emptyset$ and $\mathcal{P} \neq \mathcal{RE}$.
 - ▶ There exists at least one TM M_1 where $L(M_1) \in \mathcal{P}$.
 - ▶ There exists at least one TM M_2 where $L(M_2) \notin \mathcal{P}$.

4. Statement of Rice's Theorem

Rice's Theorem

Let $\mathcal{P}$ be any non-trivial property of Turing-recognizable languages. Then the language

$$L_{\mathcal{P}} = \{\langle M \rangle \mid L(M) \in \mathcal{P}\}$$

is **undecidable**.

- Any question regarding the semantic behavior (the language recognized) of a Turing machine cannot be algorithmically decided for all inputs.

5. Proof Construction: The Assumption

We will prove this via reduction from the Acceptance Problem (A_{TM}).

$$A_{TM} = \{\langle M, w \rangle \mid M \text{ accepts } w\}$$

- 1 Assume, for contradiction, that $L_{\mathcal{P}}$ is decidable by a TM $D_{\mathcal{P}}$.
- 2 Without loss of generality, assume the empty language $\emptyset \notin \mathcal{P}$. (If $\emptyset \in \mathcal{P}$, we consider $\overline{\mathcal{P}}$ instead).
- 3 Since $\mathcal{P}$ is non-trivial, $\exists M_1$ such that $L(M_1) \in \mathcal{P}$.

6. The Mapping Reduction ($A_{TM} \leq_m L_P$)

We construct a decider S for A_{TM} using D_P . On input $\langle M, w \rangle$: Construct a new machine M' that takes input x :

Machine $M'(x)$

1. Simulate M on w .
2. If M halts and rejects, **reject**.
3. If M halts and accepts, simulate M_1 on x .
4. If M_1 accepts x , **accept**.

7. Analyzing the Reduction

What is the language of M' ?

- **Case 1:** M accepts w .
 - ▶ M' reaches step 3 and mimics M_1 .
 - ▶ Thus, $L(M') = L(M_1) \in \mathcal{P}$.
- **Case 2:** M does not accept w .
 - ▶ M' never reaches step 3. It rejects or loops forever.
 - ▶ Thus, $L(M') = \emptyset \notin \mathcal{P}$.

8. Conclusion of the Proof

Our decider S for A_{TM} does the following:

- 1 Constructs $\langle M' \rangle$ from $\langle M, w \rangle$.
- 2 Runs the assumed decider $D_{\mathcal{P}}$ on $\langle M' \rangle$.
- 3 If $D_{\mathcal{P}}$ accepts, S accepts. If $D_{\mathcal{P}}$ rejects, S rejects.

Contradiction

We have built a decider for A_{TM} . But A_{TM} is proven to be undecidable. Therefore, our assumption is false. $L_{\mathcal{P}}$ must be undecidable. ■